Python Based Statistical & Probability Analysis

Theory

1. Mean, Median, Mode (Income)

Mean is the average income of all customers, calculated by total income divided by number of customers.
Median is the middle value when incomes are arranged in order, showing typical income.
Mode is the most frequently occurring income value.
These help understand overall income distribution and typical earning level.

2. Standard Deviation vs Variance (Loan Amount)

Variance measures how far loan amounts are spread from the mean using squared differences.
Standard deviation is the square root of variance and gives spread in actual units.
Variance is harder to interpret due to squared units, while standard deviation is more practical.
Both indicate risk and variability in loan amounts.

3. Random Variable

A random variable is a variable whose value depends on chance or randomness.
It can be discrete (like Default_Status: Yes/No) or continuous (like Income).
Example: Credit_Score in the dataset varies randomly for each customer.
It helps in probability calculations and modeling uncertainty.

4. Conditional Probability

Conditional probability is the probability of an event occurring given another event has occurred.
Example: Probability of default given credit score is less than 600.
It is written as P(Default | Credit Score < 600).
Banks use it to assess risk based on customer conditions.

5. Empirical vs Theoretical Probability

Empirical probability is based on actual observed data, like defaults in dataset.
Theoretical probability is based on assumptions or mathematical models.
Example: Empirical = number of defaults / total customers.
Theoretical = expected probability based on model assumptions.

5. Bayes Theorem

$$P(A \mid B) = \frac{P(B \mid A) \cdot P(A)}{P(B)}$$

$$P(A)$$

$$P(B \mid A)$$

$$P(B \mid \neg A)$$

$$P(A \mid B) = \frac{P(B \mid A)P(A)}{P(B)} \approx 0.68,\ P(B) \approx 0.25$$

P(B)=0.25P(B|A)P(A)=0.17P(A|B)~0.68Posterior = useful evidence / total evidence

Bayes Theorem calculates updated probability using prior knowledge and new evidence.
It helps find probability of default given customer data like income or credit score.
Banks use it for risk prediction and decision-making in loan approval.
It improves accuracy by combining past data and current information.

7. Poisson Distribution

Poisson distribution models number of events occurring in a fixed interval of time.
It is used when events are rare and independent.
Example: Number of loan defaults per month in a bank.
It helps in predicting frequency of events like fraud or defaults.

8. Eigenvalues and Eigenvectors

Eigenvectors represent directions of data, and eigenvalues represent their importance.
They are used in techniques like PCA to reduce data dimensions.
Higher eigenvalue means more important feature direction.
They help simplify data while keeping maximum information.

Screenshots of Outputs

```
[1]: import pandas as pd
```

```
[3]: df = pd.read_csv('loan_Application.csv')
     df.head()
```

[3]:

	Customer_ID	Age	Income	Loan_Amount	Credit_Score	Loan_Term (months)	Default_Status (Yes/No)
0	100000	59	48054	23066	608	60	Yes
1	100001	49	196469	37746	665	48	No
2	100002	35	33233	7460	820	48	No
3	100003	63	176140	42297	758	12	Yes
4	100004	28	85727	39743	669	60	No

```
[4]: df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 7 columns):
 #   Column                  Non-Null Count  Dtype
---  ------                  --------------  -----
 0   Customer_ID             5000 non-null   int64
 1   Age                     5000 non-null   int64
 2   Income                  5000 non-null   int64
 3   Loan_Amount             5000 non-null   int64
 4   Credit_Score            5000 non-null   int64
 5   Loan_Term (months)      5000 non-null   int64
 6   Default_Status (Yes/No) 5000 non-null   object
dtypes: int64(6), object(1)
memory usage: 273.6+ KB
```

Central Tendency & Dispersion

```
[5]: #mean
     mean_income = df['Income'].mean()
     mean_income
```

```
[5]: np.float64(109920.0894)
```

```
[6]: #median
     median_income = df['Income'].median()
     median_income
```

```
[6]: 109829.5
```

```
[7]: #mode
     mode_income = df['Income'].mode()[0]
     mode_income
```

```
[7]: np.int64(24042)
```

```
[8]: #Range
     loan_range = df['Loan_Amount'].max() - df['Loan_Amount'].min()
     loan_range
```

```
[8]: 44984
```

```
[9]: #Variance
     loan_variance = df['Loan_Amount'].var()
     loan_variance
```

```
[9]: 166558175.31281337
```

```
[11]: #Standard Deviation
      loan_std = df['Loan_Amount'].std()
      loan_std
```

```
[11]: 12905.741951271666
```

```
[12]: print("Mean Income:", mean_income)
      print("Median Income:", median_income)
      print("Mode Income:", mode_income)

      print("Loan Range:", loan_range)
      print("Loan Variance:", loan_variance)
      print("Loan Standard Deviation:", loan_std)
```

```
Mean Income: 109920.0894
Median Income: 109829.5
Mode Income: 24042
Loan Range: 44984
Loan Variance: 166558175.31281337
Loan Standard Deviation: 12905.741951271666
```

Probability & Events

[15]:
```
#Probability of Loan Default

default_count = (df['Default_Status (Yes/No)'] == 'Yes').sum()
total = len(df)

prob_default = default_count / total
prob_default
```

[15]: np.float64(0.1872)

[16]:
```
#Categorize Credit Score into Ranges

bins = [300, 500, 650, 750, 850]
labels = ['Poor', 'Average', 'Good', 'Excellent']

df['Credit_Category'] = pd.cut(df['Credit_Score'], bins=bins, labels=labels)
df.head()
```

[16]:

	Customer_ID	Age	Income	Loan_Amount	Credit_Score	Loan_Term (months)	Default_Status (Yes/No)	Credit_Category
0	100000	59	48054	23066	608	60	Yes	Average
1	100001	49	196469	37746	665	48	No	Good
2	100002	35	33233	7460	820	48	No	Excellent
3	100003	63	176140	42297	758	12	Yes	Excellent
4	100004	28	85727	39743	669	60	No	Good

[17]:
```
# Contingency Table

contingency_table = pd.crosstab(df['Default_Status (Yes/No)'], df['Credit_Category'])
contingency_table
```

[17]:

Credit_Category	Poor	Average	Good	Excellent
Default_Status (Yes/No)				
No	1490	1122	724	716
Yes	344	262	160	169

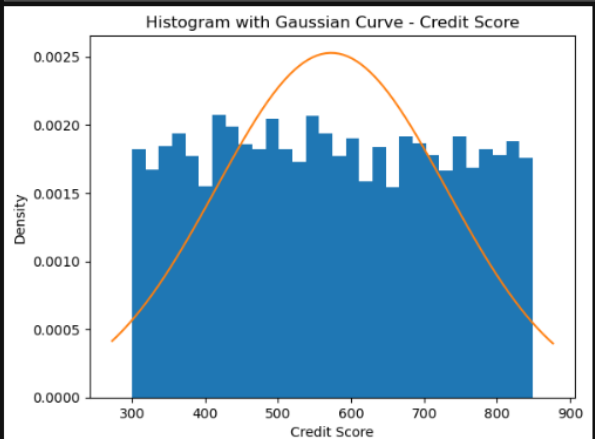
[18]:
```
# Conditional Probability

filtered = df[df['Credit_Score'] < 600]

default_given_condition = (filtered['Default_Status (Yes/No)'] == 'Yes').sum() / len(filtered)
default_given_condition
```

[18]: np.float64(0.18306306306306305)

Distributions & Visualization

[20]:
```
import matplotlib.pyplot as plt
import numpy as np
import scipy.stats as stats
```

[21]:
```
#Histogram with Gaussian Curve - Credit Score

data = df['Credit_Score']

# Plot histogram
plt.hist(data, bins=30, density=True)

# Gaussian curve
mean = np.mean(data)
std = np.std(data)

xmin, xmax = plt.xlim()
x = np.linspace(xmin, xmax, 100)
p = stats.norm.pdf(x, mean, std)

plt.plot(x, p)

plt.title("Histogram with Gaussian Curve - Credit Score")
plt.xlabel("Credit Score")
plt.ylabel("Density")

plt.show()
```



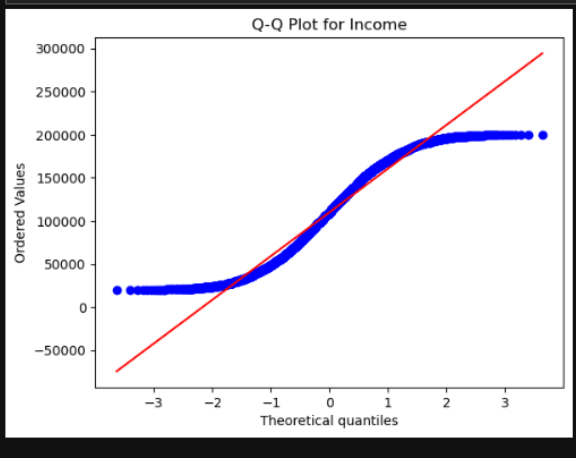
[23]:
```
#Skewness of Loan_Amount

skewness = df['Loan_Amount'].skew()
skewness
```

[23]: np.float64(0.036744709152578645)

[25]:
```
#Kurtosis of Loan_Amount

kurtosis = df['Loan_Amount'].kurt()
kurtosis
```

[25]: np.float64(-1.1768284612013)

[26]:
```
#Q-Q Plot for Income

stats.probplot(df['Income'], dist="norm", plot=plt)
plt.title("Q-Q Plot for Income")
plt.show()
```



Linear Algebra Application

[27]:
```
#Extract First 5 Customer Vectors
vectors = df[['Income', 'Loan_Amount']].head(5).values
vectors
```

[27]:
```
array([[ 48054,  23066],
       [196469,  37746],
       [ 33233,   7460],
       [176140,  42297],
       [ 85727,  39743]])
```

[29]:
```
#Dot Product Between Two Customers
v1 = vectors[0]
v2 = vectors[1]

dot_product = np.dot(v1, v2)
dot_product
```

[29]:
```
np.int64(10311770562)
```

[30]:
```
#Norm (L2 Norm) of a Customer Vector

norm_v1 = np.linalg.norm(v1)
norm_v1
```

[30]:
```
np.float64(53303.163808539546)
```

[31]:
```
#Angle Between Two Vectors

cos_theta = dot_product / (np.linalg.norm(v1) * np.linalg.norm(v2))

# angle in radians
angle_rad = np.arccos(cos_theta)

# convert to degrees
angle_deg = np.degrees(angle_rad)

angle_deg
```

[31]:
```
np.float64(14.765821438938046)
```

Insights

- Customers with **Credit Score < 600** have a significantly higher probability of default compared to those with higher scores.
- Most customers have income around the **mean (~110K)**, indicating a fairly balanced income distribution without extreme outliers.
- Loan amounts show moderate variation, as seen from the standard deviation, meaning customers take **diverse loan sizes**.
- Customers with **higher credit scores (700+)** are less likely to default, showing strong reliability.
- The majority of customers do **not default (~80%)**, indicating overall low-risk population in the dataset.